Chapter 10

HALF-EDGE DATA STRUCTURE

This chapter describes the *half-edge data structure* that forms the main solid representation of GWB.

10.1 REQUIREMENTS

Chapters 3 and 9 use plane models to discuss properties of surfaces and operations on them in a mathematically rigorous fashion. In the design of GWB, our aim will be to implement this methodology in practice. In particular, we need a data representation that can conveniently support the implementation of Euler operators.

Plane models and their manipulation operations are abstract mathematical entities, and hence very general. In particular, they include non-intuitive models (such as the skeletal plane model) that are necessary for being able to record all intermediate steps of a model description sequence.

Figure 10.1 gives some other unintuitive plane models. In particular, case (a) depicts an edge that occurs twice at a face and case (b) a polygon with just one "circular" edge whose start and end vertices are equal. As unintuitive as they are, these plane models nevertheless may act as intermediate steps during the description of plane models (c) and (d). In fact, without permitting these kinds of cases, the manipulation operations of plane models would be of quite limited use.

The inclusion all special cases becomes yet more important if we shift our attention from the domain of plane models to the domain of boundary data structures. Geometric models are not static entities that only

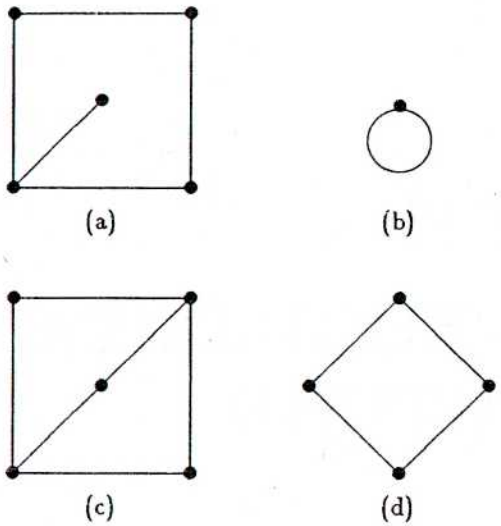

Figure 10.1 Special cases of plane models.

represent information, but data that must be worked on. For instance, a "drawing"-oriented user interface must be capable of adding new polygons, edges, and vertices to a boundary model, and it is natural that special cases will occur. Similarly, an algorithm for Boolean set operations may need to create quite unintuitive intermediate models when it manipulates boundary data structures so as to compute the desired result. Hence, all "special cases" of Figure 10.1 apply also for boundary models.

A practical boundary data structure must, of course, include also the geometric information necessary for describing a mapping from the plane model to a surface. As we shall be dealing with polyhedral models only, this mapping will be represented simply in terms of coordinates assigned to vertices. Later in Chapter 18 we shall discuss extensions to this.

10.2 OVERVIEW OF THE DATA STRUCTURE

According to Definition 3.8 on page 40, a plane model is a planar directed graph $\{N, A, R\}$ of vertices N, edges A, and polygons R. On top of the planar graph, a special topology is defined by identifying edges and vertices of individual polygons with each other. The identification allows the representation of nonplanar models.

Any data structure that aims at representing the similar information must be capable of representing both the graph structure and the identification structure of a plane model. Fortunately, according to Definition 3.9, page 41, we know that the identification structure of a realizable plane model is very simple: each edge is identified with exactly one other edge, and the polygons identified at each vertex form a single cycle. These features can be exploited in the design of the data structure.

The data structure we shall use is a variation of the full winged-edge data structure of Figure 6.7, page 109; we shall call it the *half-edge data structure*. Before going into the details of its implementation, we give first an overview to its features and its relationship to plane models in the next sections.

10.2.1 Graph Representation

For representing the graph $\{N, A, R\}$ we shall use a five-level hierarchic data structure, consisting of nodes of type *Solid*, *Face*, *Loop*, *HalfEdge*, and *Vertex*.[1] The hierarchic features of the node types are described below; the next section describes additional information that represents the identification.

Node Description

Solid Node *Solid* forms the root node of an instance of the half-edge data structure. At any point of time, several instances of the data structure may be in existence; to access any of them, a pointer to its solid node is needed. The solid node gives access to faces, edges, and vertices of the model through pointers to three doubly-linked lists. (Edge nodes are presented in the next section.) All solids are linked into a doubly-linked list realized by means of pointers to the next and the previous solid of the list.

Face Node *Face* represents one planar face of the polyhedron represented by the half-edge data structure. In GWB, a "face" is defined as a planar polygon whose interior is connected. We choose to include the convenience of faces with multiple boundaries into the data structure; hence each face is associated with a list of loops, each representing one polygonal boundary curve of the face.

As all faces represent planar polygons, one of the loops can be designated as the "outer" boundary, while others represent the "holes" of

[1] Even at the risk of confusion, we shall have "vertices" and "edges" both in the plane models and in the half-edge data structure. Which is meant should be clear from the context. The reason for the term "half-edge" is explained in due course.

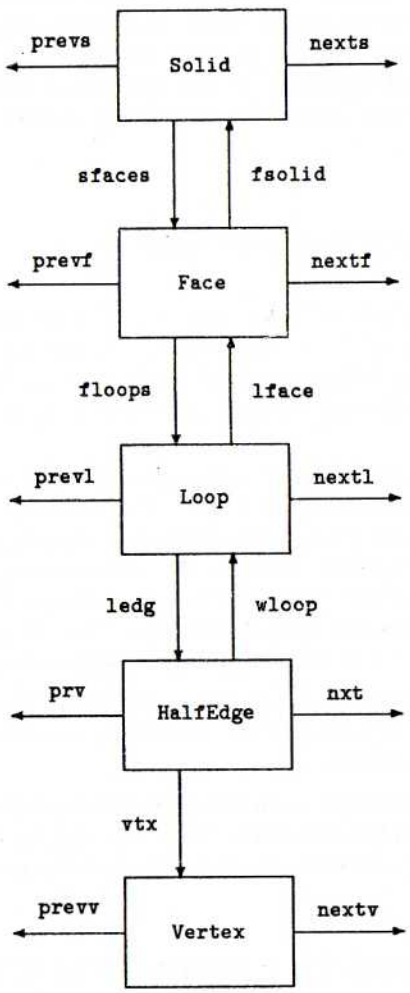

Figure 10.2 Hierarchic view of the half-edge data structure.

a face. This is realized in terms of two pointers to a loop node, one that points to the "outer" boundary, and one that points to the first loop in the doubly-linked list consisting of all loops of the face. We shall follow the convention of calling hole-loops *rings*.

Faces have a vector of four floating-point numbers that represent its plane equation. To realize the doubly-linked list of all faces of a solid, each face includes pointers to the previous and the next face in the list. Finally, each face has a pointer to its parent solid.

Loop Node *Loop* describes one connected boundary as discussed above. It has a pointer to its parent face, a pointer to one of the halfedges that form the boundary, and pointers to the next loop and to the previous loop of the face.

HalfEdge Node *HalfEdge* describes one line segment of a loop. It consists of a pointer to its parent loop and a pointer to the starting vertex of the line segment in the direction of the loop. Pointers to the previous and the next halfedge realize a doubly-linked list of halfedges of a loop; hence the final vertex of the line segment is available as the starting vertex of the next halfedge.

Vertex Node *Vertex* contains a vector of four floating-point numbers that represent a point of E^3 in the homogeneous coordinate representation explained in Appendix A. Two pointers to the next and the previous vertex realize a doubly-linked list of vertices of a solid.

The hierarchy is depicted in Figure 10.2, including the C names of some of the pointers.

10.2.2 Representation of Identification

The strict hierarchy of Figure 10.2 does not indicate directly how the individual faces are related to each other, except when several polygons refer to the same vertex node. We shall add one additional node type that makes the face-to-face relationships explicit by representing the identification of their line segments.

Recall that every edge of a valid plane model is identified with exactly one other edge. Hence, each halfedge should be associated with exactly one other halfedge. We shall use the additional node type *Edge* to record this information. As described below, we also add a few data items to certain nodes of the hierarchy to fully exploit the identification information.

Edge Node *Edge* associates two halfedges with each other; intuitively, is combines the two halves of a full edge together. It consists of pointers

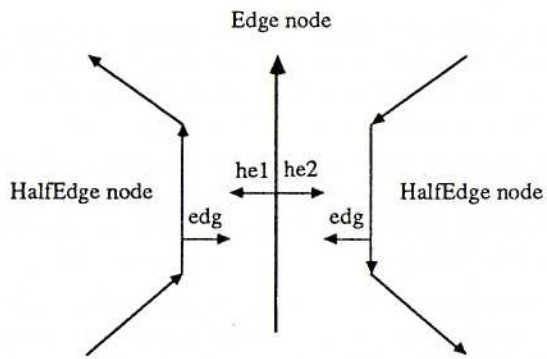

Figure 10.3 Identification of halfedges.

to the "left" and to the "right" halfedge. The doubly-linked list of edges is realized by means of pointers to the next and the previous edge.

HalfEdge Each halfedge includes an additional pointer to its parent edge.

Vertex Each vertex includes an additional pointer to one the halfedges emanating from it. To represent the identification of vertices, all faces that have a corner at the some particular point must refer (through loops and halfedges) to a single vertex node corresponding with that point.

The separation of the identification information from face boundary information brought about the division of a full winged-edge node in three nodes makes the representation of various special cases most natural; for instance, an edge occurring twice at a face would be represented as two halfedges referring to the same edge.

Observe that edges can be considered to define an orientation to its halves; that is, the "right" half is considered to be positively oriented, while the "left" half is negatively oriented.

Representation of the identification information is visualized in Figure 10.3 which also gives the C identifiers of some of the required pointers. The three nodes halfedge, edge, and vertex form the most central part of the whole data structure, and the reader is strongly urged to study these interconnections to understand how they work together.

10.2.3 Empty Loops

Following the conventions of Chapter 9, we shall allow the special case of an *empty* loop with just one vertex but no edges at all. In the half-edge data structure, empty loops will be represented in terms of one halfedge whose vertex pointer points at the single vertex, and whose edge pointer is NIL. The pointers to the next and the previous halfedge refer to this halfedge itself. Such exceptional halfedges do not correspond with line segments. Observe that empty loops cannot be naturally represented in the hierarchy of Figure 10.2.

10.3 IMPLEMENTATION DETAILS

The full C definition of the half-edge data structure is given as the include file `gwb.h` of Program 10.1. All procedures of GWB to be presented in Part Two must have a "`#include "gwb.h"`" directive that makes these definitions available, although we shall not show it explicitly in displayed algorithms.

In the definitions, the `typedef` mechanism of C is used to define the nodes of the data structure as new types. This not only leads to cleaner code, but also makes more thorough compilation time type checking possible and enhances portability. The file also defines some general-purpose types such as the type *Id* for vertex and face identifiers and the types *vector* and *matrix* for 4-vectors and 4-by-4 matrices. These types coincide with those used in the vector and matrix package of Appendix B.

The definitions themselves should be self-evident. The "generic" type *Node* is needed in the storage allocator and other low-level routines to be explained in the next chapter.

The file `params.h` referred to in Program 10.1 includes a collection of literals, macros, and global variables. Their use will be described in the following sections. For reference, the file is given in Program 10.2.

10.4 ACCESSING THE DATA STRUCTURE

The half-edge data structure is a relatively complicated object, and programs working on it tend to become quite rich of the C "arrowhead" (->) notation. This section seeks to remedy this complexity by discussing the navigation in the data structure.

```
typedef float                   vector[4];
typedef float                   matrix[4][4];
typedef short                   Id;
typedef struct solid            Solid;
typedef struct face             Face;
typedef struct loop             Loop;
typedef struct halfedge         HalfEdge;
typedef struct vertex           Vertex;
typedef struct edge             Edge;
typedef union nodes             Node;

struct solid
{
        Id          solidno;    /* solid identifier */
        Face        *sfaces;    /* pointer to list of faces */
        Edge        *sedges;    /* pointer to list of edges */
        Vertex      *sverts;    /* pointer to list of vertices */
        Solid       *nexts;     /* pointer to next solid */
        Solid       *prevs;     /* pointer to previous solid */
};
struct face
{
        Id          faceno;     /* face identifier */
        Solid       *fsolid;    /* back pointer to solid */
        Loop        *flout;     /* pointer to outer loop */
        Loop        *floops;    /* pointer to list of loops */
        vector      feq;        /* face equation */
        Face        *nextf;     /* pointer to next face */
        Face        *prevf;     /* pointer to previous face */
};
struct loop
{
        HalfEdge    *ledg;      /* ptr to ring of halfedges */
        Face        *lface;     /* back pointer to face */
        Loop        *nextl;     /* pointer to next loop */
        Loop        *prevl;     /* pointer to previous loop */
};
```

Program 10.1 C definition of the half-edge data structure.


```
struct edge
{
        HalfEdge    *he1;       /* pointer to right halfedge */
        HalfEdge    *he2;       /* pointer to left halfedge */
        Edge        *nexte;     /* pointer to next edge */
        Edge        *preve;     /* pointer to previous edge */
};
struct halfedge
{
        Edge        *edg;       /* pointer to parent edge */
        Vertex      *vtx;       /* pointer to starting vertex */
        Loop        *wloop;     /* back pointer to loop */
        HalfEdge    *nxt;       /* pointer to next halfedge */
        HalfEdge    *prv;       /* pointer to previous halfedge */
};
struct vertex
{
        Id          vertexno;   /* vertex identifier */
        HalfEdge    *vedge;     /* pointer to a halfedge */
        vector      vcoord;     /* vertex coordinates */
        Vertex      *nextv;     /* pointer to next vertex */
        Vertex      *prevv;     /* pointer to previous vertex */
};
union nodes
{
        Solid       s;
        Face        f;
        Loop        l;
        HalfEdge    h;
        Vertex      v;
        Edge        e;
};
# include "params.h"
```

Program 10.1 C definition of the half-edge data structure (cont.).

```
/* return values and misc constants */
# define        ERROR           -1
# define        SUCCESS         -2
# define        NIL             0
# define        PI              3.141592653589793

/* node type parameters for memory allocation routines */
# define        SOLID           0
# define        FACE            1
# define        LOOP            2
# define        HALFEDGE        3
# define        EDGE            4
# define        VERTEX          5

/* coordinate plane names */
# define        X               0
# define        Y               1
# define        Z               2

/* orientations */
# define        PLUS            0
# define        MINUS           1

/* macros */
# define        mate(he)        (((he) == (he)->edg->he1) ? \
                                (he)->edg->he2 : (he)->edg->he1)
# define        max(x,y)        (((x) > (y)) ? (x) : (y))
# define        abs(x)          (((x) > 0.0) ? (x) : -(x))

/* global variables */
Solid           *firsts;        /* head of the list of all solids */
extern Id       maxs;           /* largest solid no. */
extern Id       maxf;           /* largest face no. */
extern Id       maxv;           /* largest vertex no. */
extern double   EPS;            /* tolerance for geometric tests */
extern double   BIGEPS;         /* a more permissive tolerance */
```

Program 10.2 Other definitions.

10.4.1 Hierarchic Access

Many procedures need simply to scan the whole data structure, and to perform some computation at each node. This is readily possible by following the hierarchy structure of Figure 10.2. For instance, Program 10.3 gives a routine that scans all faces, loops, and halfedges of solid s, and lists the vertices of each face.

Through the hierarchy, each vertex could be listed just once by examining its halfedge pointer: if it is NIL, or equals the father halfedge, the vertex is listed; otherwise it is ignored. Similarly, each edge can be listed just once by examining its halfedge pointers. As an alternative of accessing vertices along these lines, the half-edge data structure provides a direct access path to vertices and edges.

10.4.2 Access over Edges

The hierarchic access to the half-edge data structure is sufficient for many algorithms—say, generation of graphics output and coordinate transformations. For many algorithms to be discussed in the subsequent chapters this is not sufficient, however.

Edge nodes of the data structure hold the key for more involved navigation in the data structure. In particular, from a halfedge that does not represent an empty loop, they allow the halfedge identified with it to be accessed. This task is so frequent that Program 10.2 defines a macro for it:

```
# define mate(he) (((he) == (he)->edg->he1) ?
                  (he)->edg->he2 : (he)->edg->he1)
```

In other words, if halfedge *he* is the "left" half of its edge, the "mate" halfedge is the right half, and vice versa.

This simple macro allows many apparently nontrivial procedures to be written in a straightforward manner. For instance, the procedure of Program 10.4 accesses all neighbors of a given vertex. Observe how scanning halfedges of a loop (Program 10.3) and scanning halfedges of a vertex become structurally similar to each other. As typical, "lone" vertices (belonging to an empty loop) must be handled as an exception.

10.5 NOTES

As presented, the half-edge data structure includes somewhat more data than the bare minimum. If the storage space is very scarce, it may become necessary to remove some of the redundant (but convenient) data.

```
void    listsolid(s)
Solid   *s;
{
     Face        *f;
     Loop        *l;
     HalfEdge    *he;

     f = s->sfaces;
     while(f)
     {
          printf("face %d:\n", f->faceno);
          l = f->floops;
          while(l)
          {
               printf("loop:");
               he = l->ledg;
               do
               {
                    printf(" %d: (%f %f %f)",
                           he->vtx->vertexno,
                           he->vtx->vcoord[0],
                           he->vtx->vcoord[1],
                           he->vtx->vcoord[2]);
               }
               while((he = he->nxt) != l->ledg);
               printf("\n");
               l = l->nextl;
          }
          f = f->nextf;
     }
}
```

Program 10.3 Hierarchic access.


```
void    listneighbors(v)
Vertex  *v;
{
        HalfEdge        *adj;

        adj = v->vedge;
        if(adj)
                do
                {
                        printf("%d ", adj->nxt->vtx->vertexno);
                }
                while((adj = mate(adj)->nxt) != v->vedge);
        else    printf("no adjacent vertices");
        printf("\n");
}
```

Program 10.4 Neighbor halfedge access.

The two-way linked lists of faces, edges, and vertices of a solid, and loops of a face can be changed to single-way lists with only minor modifications in the algorithms of GWB and with only small penalties in speed. As explained at the end of Section 10.4.1, the lists of edges and vertices could also be completely dropped with just a little additional overhead.

The next possible change involves the removal of back pointers from half-edges. However, this change would cause substantial revisions in the implementation of Euler operators and many of the procedures directly based on them. As a final revision, nodes *HalfEdge* and *Edge* could be merged into a single node that represents a full winged edge. Again, substantial changes would be required in the algorithms.

PROBLEMS

10.1. Learn C.

10.2. Modify the data structure definitions of Program 10.1 so that the nodes *HalfEdge* and *Edge* have been merged.

10.3. Rewrite Program 10.3 so as to use the modified data structure of Problem 10.2.

10.4. Rewrite Program 10.4 so as to use the modified data structure of Problem 10.2.

10.5. The inner loops of Programs 10.3 and 10.4 are duals of each other. How well is the dualism preserved in the modified programs of Problems 10.3 amd 10.4?

BIBLIOGRAPHIC NOTES

The data structure chosen for GWB is by no means the only possibility, and other data structures have been used to support Euler-like operations on boundary models.

For instance, Ansaldi *et al.* [4] describe a data structure called the *Face Adjacency Graph* (FAG) for representing a boundary model. The FAG treats faces as the basic nodes of the data structure, and (as the name suggests) concentrates on representing their adjacency relationships.